金

技术设计说明书

金枢 FinTrust Agent V1.0.0

面向银行业务的可信 AI 智能体系统

版本：V1.0.0 | 生成日期：2026-11-02

依据当前源码与 docs 目录编制；所有资金交易均为本地模拟

文档控制

项目	内容
文档名称	金枢 FinTrust Agent V1.0.0 技术设计说明书
事实依据	backend/、frontend/、docs/、README.md 及本次可复验命令输出
代码冻结口径	未修改核心业务代码；本文不引入新功能或新界面设计
运行模式	当前未配置 LLM_API_KEY，处于本地规则演示模式；真实模型联调未完成
资金边界	全部账户、余额、卡片、收款人、交易流水均为本地模拟，且交易记录带 is_simulated 标记

重要一致性说明：统一交付名称为“金枢 FinTrust Agent V1.0.0”；但 FastAPI /health 与根路由当前仍返回 version=0.1.0。该元数据差异不影响业务功能，已列入交付验收问题。

目录

1. 项目背景与设计目标 2. 系统整体架构 3. 模块划分 4. Agent 工作流程 5. 核心算法实现 6. 核心业务闭环 7. 数据库设计 8. API 接口设计 9. 部署环境与运行方式 10. 测试结果 11. 技术局限 12. 源码索引

1. 项目背景与设计目标

银行业务正在探索以自然语言驱动的智能服务，但大模型存在幻觉、提示词注入与越权倾向。项目把“语义理解”和“资金权限裁决”拆成两个互不信任的层次：Agent 负责把用户诉求解析为意图和结构化参数；独立安全引擎仅依据业务事实做绿/黄/红三级裁决，并贯穿确认、MFA、状态机、幂等与审计。

设计目标	实际实现	源码依据
自然语言办事	11 类意图识别与参数抽取；规则始终可用，LLM 可选增强	backend/app/agent/nlu.py
权限与模型解耦	PolicyEngine 只接收金额、当日累计、收款人属性等业务事实	backend/app/security/policy.py
写操作可解释	任何资金写操作最低黄色；规则命中列表随响应与审计返回	policy.py / transfer_service.py
执行过程可回放	TraceBuilder 记录 DAG 节点、状态、详情、耗时并落库	backend/app/agent/orchestrator.py
交易闭环真实	余额校验、条件扣款、流水、状态迁移、审计写入均落库	backend/app/services/transfer_service.py
审计可验证	HMAC 哈希链 + sequence 连续性校验 + 迁移脚本触发器	backend/app/security/audit_integrity.py
演示开箱即用	未配置模型 Key 时进入规则演示模式；演示账号由后端启动引导创建	backend/app/agent/llm.py / app/main.py

2. 系统整体架构

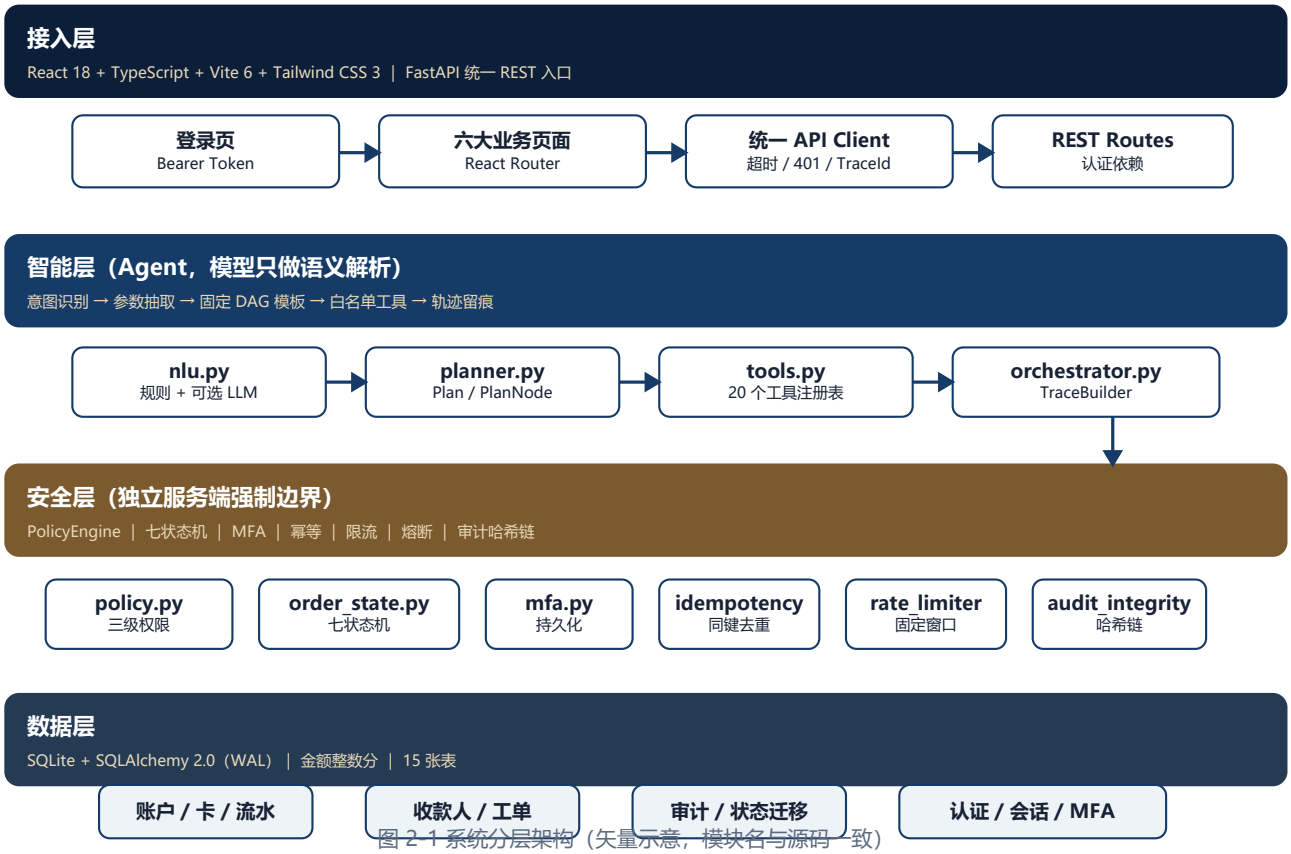


图 2-1 系统分层架构（矢量示意，模块名与源码一致）

前端通过 Vite Proxy 将 /api 转发到 127.0.0.1:8000。除登录、health、meta 以及当前实际公开的 transfers/policy、agent/capabilities 外，业务接口均要求 Bearer Token；账户归属在路由层强制校验。LLM 只参与 NLU 融合与答复润色，不参与 DAG 生成、工具选择或权限决策。

层	职责	关键边界
前端	登录、六业务页面、确认/MFA 交互、DAG 轨迹展示	仅展示与调用 API；隐藏按钮不是安全手段
路由层	认证、账户归属、限流、幂等接入、统一错误	跨账户访问 403；未认证 401

层	职责	关键边界
Agent 层	意图、参数、固定计划、工具编排、答复	工具白名单；身份上下文来自认证
安全层	三级权限、状态机、MFA、幂等、限流、熔断、审计	独立于业务和模型；不接受模型风险建议
服务层	账户、账单、卡片、转账、认证、密码	写路径调用安全层并完成数据库事务
数据层	SQLite + SQLAlchemy ORM；金额整数分	WAL；15 张核心表；只追加审计约定

3. 模块划分

3.1 后端模块

目录 / 文件	职责	关键类 / 函数
app/main.py	FastAPI 生命周期、CORS、统一异常、TraceId、路由注册、过期工单清理	create_app / lifespan
app/core/config.py	pydantic-settings 环境配置与默认值	Settings
app/core/database.py	SQLAlchemy 引擎、Session、WAL、create_all	engine / SessionLocal / init_db
app/agent/nlu.py	意图识别、中文数字金额、备注、账单 slots、LLM 融合	parse_intent / extract_amount
app/agent/planner.py	固定 DAG 模板与 DFS 三色拓扑排序	Planner.plan / Plan.topological_order
app/agent/tools.py	20 个工具注册表、账户归属断言、业务工具	tool / _assert_account_authorized
app/agent/orchestrator.py	会话、DAG 执行、答复生成、轨迹与消息落库	AgentOrchestrator.chat / TraceBuilder
app/agent/llm.py	OpenAI 兼容 HTTP 客户端、熔断包装、演示降级	LLMClient.chat_json / chat
app/security/policy.py	三级权限、动作白名单、日累计主轴、敏感词升红	PolicyEngine.evaluate
app/security/order_state.py	七状态合法迁移、终态、迁移审计	transition / allowed
app/security/mfa.py	验证码 HMAC、TTL、次数、一次性消费、绑定校验	MfaService.issue / verify
app/security/idempotency.py	幂等键、请求哈希、in_progress/done、TTL	IdempotencyStore.begin/complete/fail
app/security/rate_limiter.py	SQLite 固定窗口计数器、HMAC 脱敏键、429	check_and_increment
app/security/circuit_breaker.py	closed/open/half_open 熔断状态机	CircuitBreaker.call
app/security/audit.py	审计写入与哈希链签名	write_audit_log
app/security/audit_integrity.py	规范化内容、HMAC-SHA256、链式验证	sign_audit_log / verify_integrity
app/security/password.py	PBKDF2 密码哈希、Token HMAC 摘要	hash_password / token_hash
app/services/*	认证、账户、账单分析、卡片、转账、密码修改	transfer_service 等为交易主链路
app/api/routes/*	auth/accounts/cards/transactions/payees/transfers/agent/audit/system	REST 边界与依赖注入

3.2 前端模块

模块	职责	真实接口
src/api/client.ts	Bearer 注入、X-Trace-Id、超时、401 处理	所有 /api/v1 请求
src/api/index.ts	按业务域封装 API	authApi / transferApi / agentApi 等
src/context/AccountContext.tsx	当前账户上下文	授权账户列表
src/layouts/AppShell.tsx	侧边栏、顶栏、账户与安全设置入口	会话内导航
src/pages/Login.tsx	登录并保存 Token 与授权账户	POST /auth/login
src/pages/Dashboard.tsx	余额、预算、趋势、分类、卡片、近期流水	GET /accounts/overview、/transactions/stats
src/pages/AgentWorkbench.tsx	自然语言对话、意图/风险徽标、DAG trace 展开	POST /agent/chat、GET /agent/sessions
src/pages/BillAnalysis.tsx	周期统计、分类、关键词、分页、异常识别	GET /transactions/*
src/pages/Transfer.tsx	收款人、金额、预览、黄色确认、红色 MFA、幂等执行	POST /transfers/preview、/transfers/execute
src/pages/Cards.tsx	卡片列表、虚拟卡、冻结/解冻、挂失 MFA	GET/POST /cards/*
src/pages/Audit.tsx	审计筛选、分页、完整性验证	GET /audit、/audit/integrity

4. Agent 工作流程

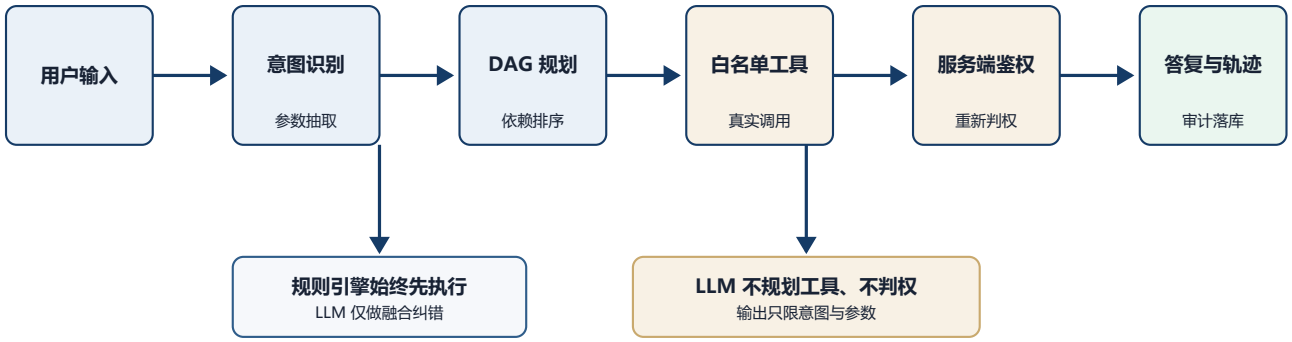


图 4-1 Agent 端到端流程

4.1 意图识别与参数抽取

parse_intent() 先执行 rule_based_intent() 与 rule_based_slots()；配置 LLM 后再调用 llm_intent()。两者意图一致时合并 slots，source 为 llm+rule；不一致时仅在 LLM 置信度至少高出 0.15 时采用 LLM 意图。LLM 不可用时自动返回规则结果，Agent 不因模型故障而不可用。

能力	实现细节
意图集合	transfer、bill_analysis、card_query、card_virtual_apply、card_lost、card_freeze、account_overview、payee_query、audit_query、help、unknown
金额解析	支持阿拉伯数字、万、中文数字/大写数字及货币单位；排除“尾号/卡号/账号”等非金额前缀
转账参数	amount、payee_name、payee_account_no、card_last4、remark
账单参数	category、period、direction、focus；支持本月/上月/近七天等口径
LLM 输出约束	仅允许白名单意图；slots 必须为 dict；confidence 限幅到 0~1

4.2 DAG 规划与执行轨迹

Planner 按意图选择服务端固定模板。例如转账 DAG 为 resolve_payee → check_daily_total → evaluate_policy → prepare_order；账单分析 DAG 并行拉取流水后聚合分类、识别异常、计算趋势，再由 compose_summary 汇合。Plan.topological_order() 使用 DFS 三色访问，环回边被跳过以避免死循环；这不是 Kahn 算法。

Orchestrator 对每个节点检查工具调用次数（默认 10）和整体耗时（默认 30 秒），通过 get_tool() 白名单取得函数并注入 ctx。TraceBuilder 记录节点名、标题、状态、详情、耗时与 payload，最终以 JSON 保存到 agent_messages.trace_json，前端据此展示真实执行轨迹。

4.3 工具调用边界

tools.py 通过装饰器注册 20 个工具。编排器不接受模型动态生成的工具名；未知工具会被 skipped/failed。ctx[auth] 中的 actor、user_id 与 authorized_account_ids 来自路由认证上下文，不由 LLM 提供。实际资金执行仍走 transfer_service/card_service 的策略引擎与状态机。

5. 核心算法实现

5.1 三级安全权限判定

等级 / 决策	代码条件	行为
GREEN / allow	ActionType 属于 READ_ONLY_ACTIONS	只读查询自动执行，仍写审计
YELLOW / confirm	CONFIRMABLE_ACTIONS；转账执行后日累计 ≤ ¥1,000	返回完整明细，要求用户确认；HTTP 409
RED / mfa	日累计 > ¥1,000；挂失/改密；单笔 ≥ ¥10,000；备注敏感词	强制模拟 MFA；HTTP 428
RED / block	执行后日累计 > ¥100,000 或未知动作	直接阻断；HTTP 403

PolicyEngine.evaluate() 的入参只包含 amount_cents、daily_total_cents、收款人属性与敏感词等业务事实；模型输出中的 risk、authorized、actor 等字段不会传入策略引擎。每次判定生成 rules 列表，供前端展示和审计落库。

5.2 日累计金额计算

daily_transfer_total() 按业务时区（默认 Asia/Shanghai，缺 tzdata 时回退 UTC+8）计算当前自然日，在 UTC 起止窗口内对 transactions.amount_cents 做 SUM。过滤条件为 account_id、direction=expense、category=转账支出、occurred_at 位于当日窗口。金额在传输、存储、扣减中均使用整数分。

5.3 七状态交易状态机

合法迁移由 order_state._ALLOWED 统一约束；四个终态无出边

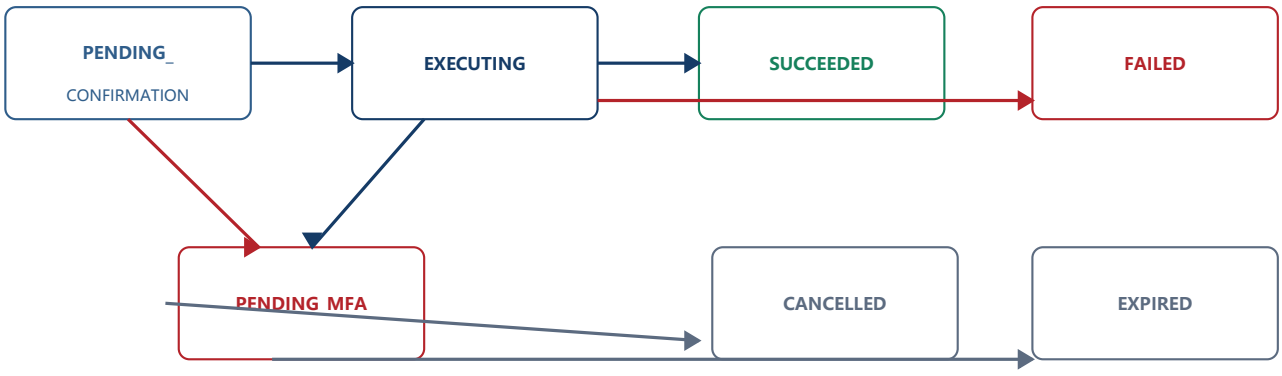


图 5-1 七状态交易状态机与主要迁移

状态迁移只能经 transition() 完成；_ALLOWED 表定义 PENDING_CONFIRMATION、PENDING_MFA、EXECUTING 及四个终态的合法出边。迁移会更新 version 与时间戳，并只追加写入 order_status_transitions。相同状态迁移视为幂等直接返回。

5.4 执行时重新判权

execute_transfer() 不信任预览快照：执行时重新读取账户、重算日累计并调用 PolicyEngine。若执行时风险上升，返回 428 且不扣款；结合预览 requires_mfa 保证“只升不降”。MFA 通过后还会再做一次完整重判，防止验证窗口内累计变化。

5.5 幂等与并发

IdempotencyStore 以 (key, endpoint) 唯一约束保存请求 SHA-256、响应和 in_progress/done 状态。同键同参返回首次响应，同键异参返回冲突；并发插入命中唯一约束后重查。当前实际接入 POST /transfers/execute；请求 Header
在路由层虽声明为可空，但前端会为执行请求生成稳定幂等键，后端还有成功工单幂等返回兜底。

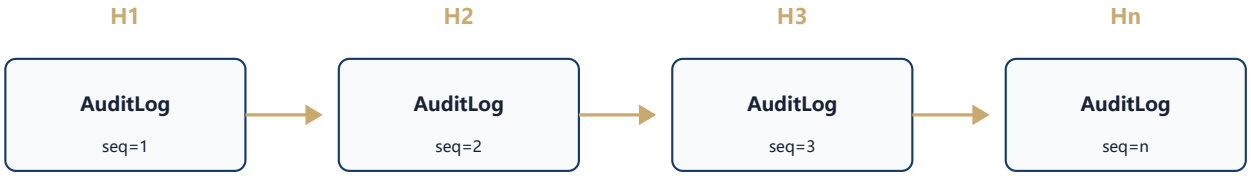
黄色转账路径通过 BEGIN IMMEDIATE 串行化写事务，余额扣减使用条件 UPDATE（balance_cents >= amount）并在同一事务中完成 EXECUTING、扣款、流水和 SUCCEEDED。红色路径在 mfa_service.verify() 会提交挑战消费，后续扣款阶段不再由同一 IMMEDIATE 事务覆盖；该边界已列入安全自评残余风险，不能宣称所有 MFA 并发场景均已被测试覆盖。

5.6 MFA 验证

MfaService.issue() 使用 secrets 生成 6 位验证码，仅保存 HMAC-SHA256(code, MFA_SECRET_KEY)。挑战记录包含 user/account/operation/target 绑定、300 秒 TTL、5 次错误尝试与一次性消费状态。verify() 依次检查存在性、消费、过期、锁定、绑定与摘要（hmac.compare_digest），成功后置为 consumed。

演示模式下响应会返回明文 demo_code，属于明确标注的模拟 MFA；正式模式不返回明文。转账与挂失当前发放挑战时未传 user_id，验证主要绑定账户、操作和目标；密码修改完整绑定用户。

5.7 审计哈希链



event_hash = HMAC-SHA256(key, previous_hash + canonical(event))；sequence 必须连续

图 5-2 审计哈希链结构

sign_audit_log() 读取最近一条 sequence_id 非空的审计记录作为前驱，计算 next_seq 与 event_hash；verify_integrity() 按 sequence 升序验证连续性、event_hash 与 previous_hash，并在失败时返回首个异常记录的 sequence_id 和 audit_id。canonical 序列化固定字段与 JSON 键序，保证同一事件哈希确定。

迁移脚本 migrate_v040.py 会创建阻止已签名 audit_logs UPDATE/DELETE 的触发器；全新 create_all 路径不会自动创建该触发器，应按部署说明补跑迁移。

6. 核心业务闭环：智能转账

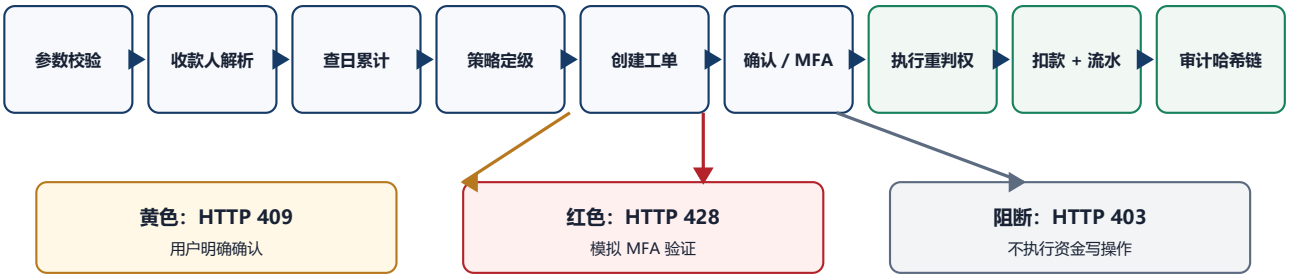


图 6-1 转账业务闭环

阶段	真实处理	结果 / 留痕
查询	读取收款人、账户、当日累计与策略阈值	页面展示；不改动资金
预览	参数校验、收款人解析、日累计、PolicyEngine 定级	创建 transfer_orders；写 transfer.preview 审计
风险判断	只读绿；小额写黄；累计超限/敏感词红；超红线阻断	返回 200/409/428/403 及可解释 rules
确认与 MFA	黄色要求 confirm=true；红色要求 challenge_id + 6 位验证码	MFA_REQUIRED 写审计；错误验证码写 mfa_failed
执行	重新判权 → EXECUTING → 条件扣 balance/available → 写流水 → SUCCEEDED	事务原子提交；失败独立事务落 FAILED
余额与流水	Transaction 记录转账支出、balance_after、order_no、is_simulated	前端返回 transaction_id 与余额
审计	transfer.executed / cancelled / policy_blocked / mfa_* 写入哈希链	GET /audit/integrity 可验证

系统不会伪造成功：余额不足、验证码错误、累计超限、工单过期等均返回明确错误；只有真实完成扣款与流水落库后才返回 SUCCEEDED 和 transaction_id。

账单分析闭环

Agent 或页面调用 transactions/stats，analytics_service 返回收支汇总、分类占比、月度趋势与异常列表。异常规则在代码中包括分类基线倍数、凌晨时段和夜间高额等；展示口径以 analytics_service.py 当前实现为准。

银行卡管理闭环

卡片查询为绿色；虚拟卡申请与冻结/解冻最低黄色并需确认；挂失强制红色 MFA。卡片状态变更在 card_service 中执行并返回明确状态；MFA 挑战持久化于 mfa_challenges。

7. 数据库设计

数据库使用 SQLite + SQLAlchemy 2.0，开启外键与 WAL。金额字段统一为 *_cents 整数；演示交易、卡片与工单均带 is_simulated 标记。当前核心数据表共 15 张。

表	核心字段	关系 / 说明
accounts	account_no、holder_name、account_type、balance_cents、available_cents、status	账户主表；一对多卡片、流水、收款人
cards	account_id、card_no、card_type、status、daily_limit_cents、is_virtual、is_simulated	账户卡片；删除账户级联
transactions	account_id、card_id、direction、category、amount_cents、balance_after_cents、status、occurred_at、is_simulated	交易流水；账户/时间索引
payees	account_id、name、account_no、is_frequent、single_limit_cents、total_count、last_used_at	收款人；归属账户
audit_logs	actor、action、target、risk_level、decision、reason、detail_json、ip_address、sequence_id、previous_hash、event_hash	只追加审计 + 哈希链
agent_sessions	session_id、title、account_id、message_count	会话主表
agent_messages	session_id、role、content、intent、confidence、risk_level、trace_json、demo_mode	会话消息；trace JSON
transfer_orders	order_no、account_id、payee_id、amount_cents、risk_level、status、policy_json、mfa_challenge_id、version、各时间戳	转账工单；七状态生命周期
order_status_transitions	order_no、from_status、to_status、actor、reason、trace_id	状态迁移只追加审计
idempotency_records	key、endpoint、request_hash、response_json、status	(key, endpoint) 唯一
users	username、password_hash、status	认证用户
user_accounts	user_id、account_id、permission	用户账户授权；user+account 唯一
sessions	user_id、token_hash、expires_at、revoked_at	服务端仅存 Token 摘要
mfa_challenges	challenge_id、user_id、account_id、operation、target_id、code_hash、expires_at、attempts_remaining、status	持久化 MFA 挑战
rate_limit_records	key、window_start、count	限流窗口；(key, window_start) 唯一

初始化与迁移

脚本	用途	注意
scripts/init_db.py	create_all、种子数据、reset、recreate	--recreate/--reset 会清空业务数据；仅用于全新环境
scripts/migrate_v030.py	认证、七状态机及新旧状态映射	旧库升级不重置余额流水
scripts/migrate_v040.py	审计哈希链字段、限流表、审计触发器	create_all 新库也需补跑才有触发器
scripts/prepare_demo.py	演示前重置演示库当日累计并输出检查清单	须在数据库和用户引导完成之后运行
scripts/verify_audit_integrity.py	命令行审计完整性验证	返回 PASS/FAILED

8. API 接口设计

统一前缀 /api/v1；统一错误格式为 error.code/message/detail/trace_id。认证使用 Authorization: Bearer；前端自动生成 X-Trace-Id。写操作前端会携带 Idempotency-Key。

方法 / 路径	认证与归属	核心语义
POST /auth/login	公开	用户名密码认证，返回 Token、有效期与授权账户
POST /auth/logout、GET /auth/me	认证	撤销会话；返回当前用户与账户
GET /accounts、GET /accounts/overview	认证 + 账户归属	授权账户列表与账户总览
GET /cards、GET /cards/{id}	认证 + 归属	卡片查询（绿色）
POST /cards/virtual、POST /cards/freeze、POST /cards/lost	认证 + 归属	黄色确认 / 红色 MFA
GET /transactions、/stats、/categories、/anomalies	认证 + 归属	流水、账单统计、分类、异常识别
GET /payees、POST /payees	认证 + 归属	收款人查询与新增
GET /transfers/policy、GET /transfers/daily-total	当前实际：policy 公开；daily-total 需认证	阈值说明与当日累计
POST /transfers/preview	认证 + 归属	创建工单、服务端定级、写审计
POST /transfers/execute	认证 + 工单归属	确认/MFA/重判权/执行；接入幂等存储
POST /transfers/{order_no}/cancel、GET /transitions	认证 + 归属	取消与状态迁移查询
POST /agent/chat、GET /agent/sessions*、GET /agent/capabilities	chat/sessions 需认证；capabilities 当前公开	Agent 对话、轨迹、会话与能力清单
GET /audit、GET /audit/integrity	认证；审计按 actor 隔离	审计筛选分页与哈希链验证
POST /accounts/password/change/challenge、/accounts/password/change	认证	红色 MFA 改密；成功后撤销全部会话
GET /health、GET /meta/simulation-notice	公开	服务健康与模拟交易声明

当前 /transfers/policy 与 /agent/capabilities 实际未要求认证，与 docs/API.md “除 health/meta/login 外全部认证”的表述不完全一致。本文以代码为准。

9. 部署环境与运行方式

9.1 环境要求

组件	要求	本次验证
操作系统	Windows / macOS / Linux 本地运行	Windows
Python	3.11+（项目实测 3.13.14）	backend/.venv
Node.js	18+（项目实测 22.22.2）	frontend/node_modules 已安装
数据库	SQLite；无需外部服务	backend/data/*.db
模型	OpenAI 兼容接口；未配置时使用规则演示模式	当前 llm_configured=false

9.2 从零启动

后端：python -m venv .venv → pip install -r requirements.txt → python scripts/init_db.py --recreate（仅全新环境）→ 启动一次后端完成演示用户引导 → 可选 migrate_v030.py / migrate_v040.py → uvicorn app.main:app --port 8000。前端：npm install → npm run dev。Vite 将 /api 代理到 127.0.0.1:8000。Windows 根目录提供 start_backend.bat 与 start_frontend.bat。

9.3 独立部署所需文件

源码包必须包含 backend/app、backend/tests、backend/scripts、requirements.txt、pytest.ini、run.py、backend/.env.example，frontend/src、package.json、package-lock.json、Vite/TS/Tailwind 配置、frontend/.env.example，docs/ 全部说明，以及根 README.md 与启动脚本。真实 .env、API Key、Token、数据库文件、虚拟环境和 node_modules 均不得进入源码包。

9.4 生产化边界

当前提供的是本地/竞赛部署路径，没有 Dockerfile、Nginx、HTTPS、进程守护或分布式限流。跨源部署需要配置 CORS 与 VITE_API_BASE_URL，并自行增加静态资源托管与 TLS。

10. 测试结果与证据

验证项	实际命令 / 时间	本次结果
后端测试	backend/.venv/Scripts/python.exe -m pytest backend/tests -q --disable-warnings（2026-11-02 22:39 复跑）	117 个用例全部通过
前端生产构建	npm --prefix frontend run build（2026-11-02 22:38）	成功；TypeScript 构建通过，1597 个模块
服务健康	GET /api/v1/health	database=connected；demo_mode=true；llm_configured=false
真实页面	本地 5173 登录并截取登录、总览、Agent、转账、审计页面	截图均来自实际运行系统
审计完整性	前端调用 GET /audit/integrity	AUDIT INTEGRITY: PASS；13 条已签名事件

本次交付没有重新生成覆盖率报告，也没有连续 10 次测试日志；因此文档只确认上述可复验结果。docs/FINAL_TEST_REPORT.md 中的历史覆盖率与“连续 10 次零失败”不作为本次独立复证结论。

测试覆盖范围（源码测试集）

测试文件	静态收集	主要覆盖
test_policy.py	12	绿黄红、日累计边界、敏感词、白名单
test_agent_nlu.py	10	金额参数化、意图、备注抽取
test_api.py	16	API、转账、认证、Agent
test_p0_security.py	12	执行时重判、20 线程黄色并发、幂等、挂失
test_p1.py	32	状态机、认证、MFA、密码修改
test_p2_rate_limit.py	6	登录、Agent、预览、MFA 限流
test_p2_agent_security.py	16	注入、伪造字段、白名单、工具边界
test_p2_audit_integrity.py	9	哈希链、篡改、中间删除、触发器
合计	117	以 pytest 实际收集结果为准

11. 技术局限与已知问题

类别	事实与影响	当前状态
真实 LLM	未配置 Key，真实调用、成功率和多轮效果未联调；当前为规则演示模式	如实限制
多轮上下文	_compose_reply 只传当前 message，历史消息未进入 NLU/答复	未实现
NLU 澄清	IntentResult 缺少 missing_parameters / clarification_required	未实现
限流	SQLite 固定窗口 + 进程内状态，适合单实例竞赛演示	非分布式生产级
熔断	HALF_OPEN 被 is_open 判定为开放，正常探测恢复路径存在缺陷	已知代码问题
MFA 并发	红色路径验证挑战会提交并释放前序写锁，未再获取 IMMEDIATE 锁	残余并发边界
幂等接入	幂等存储实际只接入转账执行；Header 缺失时退化为工单号去重	写接口覆盖不完整
审计尾部	最后一条或全部已签名记录删除可能不产生 sequence 缺口；未签名记录不参与验证	检测能力边界
审计原子性	业务扣款提交与审计签名不是同一事务，极端失败窗口可能留下业务已完成但审计未完成	诚实声明
Agent 工具审计	list_audit 工具查询未按当前 actor 过滤；Agent 会话复用缺少账户一致性核对	已知授权缺口
元数据	FastAPI health/root 仍返回 0.1.0；frontend package-lock 根版本与 package.json 不一致	不影响业务，待统一
文档历史口径	个别早期 docs 阈值、测试数量、算法描述与当前代码不一致	本文以源码为准

12. 源码索引（事实核对路径）

主题	路径
应用入口 / 配置 / 数据库	backend/app/main.py；backend/app/core/config.py；backend/app/core/database.py
Agent	backend/app/agent/{nlu.py,planner.py,tools.py,orchestrator.py,llm.py}
安全	backend/app/security/{policy.py,order_state.py,mfa.py,idempotency.py,rate_limiter.py,circuit_breaker.py,audit.py,audit_integrity.py,password.py}
业务	backend/app/services/{transfer_service.py,analytics_service.py,card_service.py,auth_service.py,password_service.py,account_service.py}
接口	backend/app/api/routes/*.py；docs/API.md
数据模型	backend/app/models/__init__.py
脚本 / 测试	backend/scripts/*.py；backend/tests/*.py
前端	frontend/src/pages/*.tsx；frontend/src/api/*.ts；frontend/src/layouts/AppShell.tsx
部署	README.md；docs/DEPLOYMENT.md；start_backend.bat；start_frontend.bat

结论：金枢 FinTrust Agent V1.0.0 的前后端、Agent、安全引擎与数据库链路完整可运行；核心权限判定与模型解耦，资金写路径具备确认/MFA、状态机、幂等与审计闭环。本报告同时保留真实 LLM 未联调、单机限流、哈希链检测边界及上述残余问题，不作超出代码能力的声明。